МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гомельский государственный технический университет имени П.О. Сухого»

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине: «ООПиП»
на тему: «Обобщенные коллекции, итераторы и возможности *LINQ* в языке *C#*»

Выполнил: студент гр. ИТП-21
Лосев М.А.
Принял: ассистент
Карась О.В.

Гомель 2026

Цель работы: Изучение принципов построения фундаментальных структур данных, реализации стандартных интерфейсов *.NET* (*ICollection<T>* , *IEnumerable<T>*) и освоение технологии *LINQ* для фильтрации, преобразования и динамической обработки данных в кастомных контейнерах.

Задание:

1. Разработать обобщенную (*Generic*) коллекцию согласно варианту. Запрещено использовать стандартные коллекции (*List*, *Stack* и др.) для внутреннего хранения. Использовать массивы или узловые структуры.

2. Создать иерархию (Базовый абстрактный класс + минимум 2 наследника). Классы должны содержать не менее 3 свойств (строка, число, дата/логическое значение).

3. Реализовать *ICollection<T>* и *IEnumerable<T>* (через *yield return* или свой класс-итератор).

4. Работа с *LINQ*:

– Реализовать не менее 5 различных операций (Фильтрация, Сортировка по двум полям, Группировка, Агрегация (*Sum*/*Average*/*Max*), Проекция в новый тип).

– Каждая операция должна быть продублирована в двух форматах: *Query Syntax* и *Method Syntax*.

– Продемонстрировать преобразование форматов: например, результат *LINQ*-запроса (*IEnumerable* или анонимный тип) преобразовать в массив или новую коллекцию вашего типа.

5. Реализовать графический интерфейс (*WPF*/*WinForms*), который позволяет:

– Визуализировать структуру коллекции.

– Динамически добавлять фильтры

– формирует итоговый запрос.

– Отображать результаты в отдельной таблице или списке.

6. Построить *UML*-диаграмму классов системы.

1. Описание реализации:

Разработана обобщённая коллекция *MyLinkedList<T>* на основе односвязного списка, реализующая интерфейсы *ICollection<T>* и *IEnumerable<T>*.

Создана иерархия классов:

– *AssemblyStage* (базовый) – свойства: *Name*, *DurationMinutes*, *StartDate*, *Workshop*, *Danger*.

– *ManualStage* – добавлено *RequiresCertification*.

– *AutomatedStage* – добавлено *MachineId*.

– Создан менеджер *StageManagerQueryable*, реализующий интерфейс *IStageManager*, который обеспечивает:

– фильтрацию элементов (*GetFilteredStages*),

– расчет среднего времени этапов для конкретного цеха (*CalculateAverage*),

– группировку по уровню опасности (*GroupByDanger*).

– Для каждой операции использованы два синтаксиса *LINQ*: *Query Syntax* и *Method Syntax*.

Результаты *LINQ* могут быть преобразованы в массив или новую коллекцию *MyLinkedList<T>*.

2. Графический интерфейс *WPF*:

– Левая колонка – фильтры и действия, включая расчет среднего и группировку.

– Средняя колонка – исходные данные в виде *JSON*.

– Правая колонка – результаты фильтров и группировок.

– Фильтры могут применяться по отдельности или совместно, результаты отображаются сразу после нажатия кнопки.

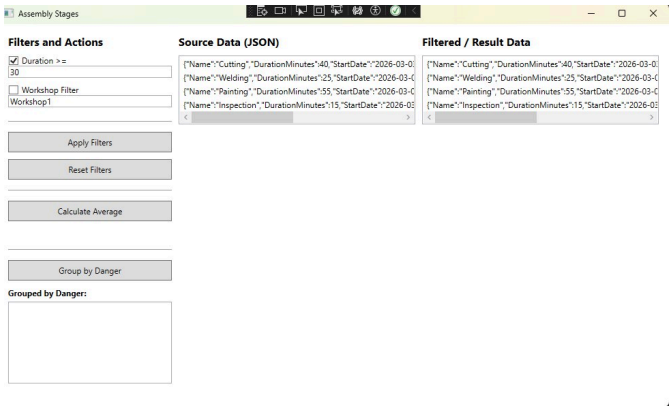

Рисунок 1 – Интерфейс программы

3. *UML* – диаграмма классов системы

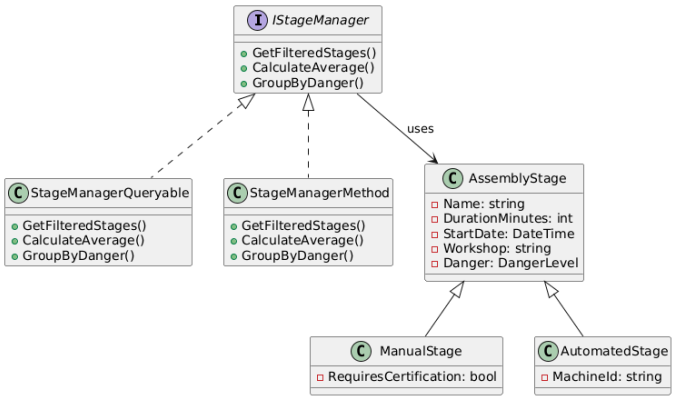

Рисунок 2 – *UML* – диаграмма классов системы

Диаграмма отображает:

– интерфейс *IStageManager* с методами фильтрации, расчета среднего и группировки,

– классы менеджеров *StageManagerQueryable* и *StageManagerMethod*,

– базовый класс *AssemblyStage* и наследников *ManualStage* и *AutomatedStage* с их свойствами,

– связи наследования и использования данных менеджером.

Вывод: Работа продемонстрировала создание собственной коллекции, реализацию интерфейсов *.NET*, применение *LINQ* для фильтрации, проекции и группировки, а также визуализацию данных через *WPF*. Все операции корректно работают и результаты совпадают между разными реализациями менеджера.

ПРИЛОЖЕНИЕ А
(обязательное)

MainWindow.xaml.cs

```
using System.Windows;
using System.Text.Json;
using Second.ClassLib.First;
using Second.Wpf.First.StageManagers;

namespace Second.Wpf.First;

public partial class MainWindow : Window
{
    private readonly MyLinkedList<AssemblyStage> data = new
MyLinkedList<AssemblyStage>();
    private readonly IStageManager manager = new StageManagerQueryable();

    public MainWindow()
    {
        InitializeComponent();

        SeedData();
        RefreshSourceView();
        RefreshResultView(data);
    }

    private void SeedData()
    {
        data.Add(new ManualStage("Cutting", 40, DateTime.Now, "Workshop1",
DangerLevel.High, true));
        data.Add(new AutomatedStage("Welding", 25, DateTime.Now, "Workshop2",
DangerLevel.Medium, "M-01"));
        data.Add(new ManualStage("Painting", 55, DateTime.Now, "Workshop1",
DangerLevel.Low, false));
        data.Add(new AutomatedStage("Inspection", 15, DateTime.Now, "Workshop3",
DangerLevel.Low, "M-02"));
    }

    private void RefreshSourceView()
    {
        SourceList.ItemsSource = data.Select(ToJson).ToList();
    }

    private void RefreshResultView(IEnumerable<AssemblyStage> result)
    {
        ResultList.ItemsSource = result.Select(ToJson).ToList();
    }

    private static string ToJson(AssemblyStage stage)
    {
        return JsonSerializer.Serialize(stage, new JsonSerializerOptions
{ WriteIndented = false });
    }

    private void Apply_Click(object sender, RoutedEventArgs e)
    {
        var query = data.AsEnumerable();
```

```
        if (UseDurationFilter.IsChecked == true && int.TryParse(DurationBox.Text,
out int minDuration))
            query = manager.GetStagesLongerThan(query, minDuration);

        if (UseWorkshopFilter.IsChecked == true && !
string.IsNullOrWhiteSpace(WorkshopBox.Text))
            query = query.Where(s => s.Workshop == WorkshopBox.Text);

        RefreshResultView(query);
    }

    private void Reset_Click(object sender, RoutedEventArgs e)
    {
        UseDurationFilter.IsChecked = true;
        UseWorkshopFilter.IsChecked = false;
        DurationBox.Text = "30";
        WorkshopBox.Text = "";
        AverageResult.Text = "";
        GroupResult.ItemsSource = null;

        RefreshResultView(data);
    }

    private void Average_Click(object sender, RoutedEventArgs e)
    {
        if (string.IsNullOrWhiteSpace(WorkshopBox.Text))
        {
            MessageBox.Show("Enter a workshop to calculate average");
            return;
        }

        double avg = manager.GetAverageDurationByWorkshop(data, WorkshopBox.Text);
        AverageResult.Text = $"Average duration in {WorkshopBox.Text}: {avg:F2}
min";
    }

    private void Group_Click(object sender, RoutedEventArgs e)
    {
        var groups = manager.GroupByDangerLevel(data);

        GroupResult.ItemsSource = groups
            .Select(g => $"{g.Key}: {g.Count()} stage(s)")
            .ToList();
    }
}
```